K R Pete Krishna Government Engineering College
K R Pete, Mandya District-571426
Department of Electronics and Communication Engineering

VLSI DESIGN AND TESTING

LAB MANUAL

COURSE CODE: BECL606

VI SEMESTER

Academic Year: 2024-25

VTUSYNC.IN

Course Material Prepared by:

Dr. Rangaraju H G
Professor
Department of Electronics and Communication Engineering

VLSI Design and Testing LAB			
Course Code	**BECL606**	CIE Marks	50
Teaching Hours/Week (L: T: P: S)	0:0:2:0	SEE Marks	50
Credits	1	Exam Hours	3

Course objectives:

This laboratory course enables students to

- Design, model, simulate and verify digital circuits.
- Perform ASIC design flow and understand the process of synthesis, synthesis constraints and evaluating the synthesis reports to obtain optimum gate level netlist.
- Perform RTL-GDSII flow and understand the ASIC Design flow.

Sl. No	Experiments
1	**Design a 4-Bit Adder** • Write a Verilog description • Verify the Functionality using Test-bench • Synthesize the design by setting proper constraints and generate the gate level netlist. From the report generated identify Critical path, Maximum delay, Total number of cells, Power requirement and Total area required
2	**4-Bit Shift and add Multiplier** • Write Verilog Code • Verify the Functionality using Test-bench • Synthesize the design by setting proper constraints and obtain the gate level netlist. From the report generated identify Critical path, Maximum delay, Total number of cells, Power requirement and Total area required
3	**32-Bit ALU Supporting 4-Logical and 4-Arithmetic operations, using case and if statement for ALU Behavioral Modeling** • Write Verilog description • Verify functionality using Test-bench • Synthesize the design targeting suitable library and by setting area and timing constraints • Tabulate the Area, Power and Delay for the Synthesized netlist • Identify Critical path
4	**Flip-Flops (D, SR and JK)** • Write the Verilog description • Verify the Functionality using Test-bench • Synthesize the design by setting proper constraints and obtain the gate level netlist. From the report gate level netlist identify Critical path, Maximum delay, Total number of cells, Power requirement and Total area required. • Verify the functionality using Gate level netlist and compare the results at RTL and gate level netlist.
5	**Four-bit Synchronous MOD-N counter with Asynchronous reset** • Write Verilog Code • Verify functionality using Test-bench • Synthesize the design targeting suitable library and by setting area and timing constraints • Tabulate the Area, Power and Delay for the Synthesized netlist Identify Critical path

VTUSYNC.IN

	• Verify the functionality using Gate level netlist and compare the results at RTL and gate level netlist.
6	a) **Construct the schematic of CMOS inverter** with load capacitance of 0.1pF and set the widths of inverter with Wn = Wp, Wn = 2Wp, Wn = Wp/2 and length at selected technology. Carry out the following: i. Set the input signal to a pulse with rise time, fall time of 1ns and pulse width of 10ns and the time period of 20ns and plot the input voltage and output voltage of designed inverter? ii. From the simulation result compute tp_{HL}, tp_{LH} and td for all three geometrical settings of width? iii. Tabulate the results of delay and find the best geometry for minimum delay for CMOS inverter. b) **Draw layout of inverter with Wp/Wn = 40/20**, use optimum layout methods. Verify for DRC and LVS, extract parasitic and perform post layout simulations, compare the results with pre layout simulations and compare the results.
7	**Capture the schematic of 2-input CMOS NOR** gate having similar delay as that of CMOS inverter computed in experiment above. Verify the functionality of NOR gate and also find out the delay td for all four possible combinations of input vectors. Table the results. Increase the drive strength to 2X and 4X and tabulate the results.
8	**Construct the schematic of the Boolean Expression** $Y = \overline{AB+CD+E}$ using CMOS Logic. Verify the functionality of the expression find out the delay td for some combination of input vectors. Tabulate the results.
9	a) Construct the schematic of Common Source Amplifier with PMOS Current Mirror Load and find its transient response and AC response? Measure the Unit Gain Bandwidth (UGB), amplification factor by varying transistor geometries, study the impact of variation in width to UGB. b) Draw Layout of common source amplifier, use optimum layout methods. Verify for DRC & LVS, extract parasitic and perform post layout simulations, compare the results with pre-layout simulations. Record the observations.
10	a) Construct the schematic of two-stage operational amplifier and measure the following: i. Unity gain Bandwidth ii. dB Bandwidth iii. Gain Margin and phase margin with and without coupling capacitance iv. Use the op-amp in the inverting and non-inverting configuration and verify its functionality. v. Study the UGB, 3dB bandwidth, gain and power requirement in op-amp by varying the stage wise transistor geometries and record the observations. b) Draw layout of two-stage operational amplifier with minimum transistor width set to 300 (in 180/90/45 nm technology), choose appropriate transistor geometries as per the results obtained in part a. Use optimum layout methods. Verify for DRC and LVS, extract parasitic and perform post layout simulations, compare the results with pre-layout simulations and perform the comparative analysis.
	Demonstration Experiments (For **CIE**)
11	**UART** • Write Verilog description • Verify the Functionality using Test-bench • Synthesize the design targeting suitable library and by setting area and timing constraints • Tabulate the Area, Power and Delay for the Synthesized netlist, Identify Critical path

VTUSYNC.IN

12	**Design and characterize 6T binary SRAM cell and measure the following:** • Read Time, Write Time, SNM, Power • Draw Layout of 6T SRAM, use optimum layout methods. Verify for DRC & LVS, extract parasitic and perform post layout simulations, compare the results with pre-layout simulations. Record the observations.

Course outcomes (Course Skill Set):

On the completion of this laboratory course, the students will be able to:

1. Design and simulate combinational and sequential digital circuits using Verilog HDL.
2. Understand the synthesis process of digital circuits using EDA tool.
3. Perform ASIC design flow and understand the process of synthesis, synthesis constraints and evaluating the synthesis reports to obtain optimum gate level netlist.
4. Design and simulate basic CMOS circuits like inverter, NOR gate and any Boolean expression.
5. Perform RTL_GDSII flow and understand the stages in ASIC design.

Assessment Details (both CIE and SEE)

The weightage of Continuous Internal Evaluation (CIE) is 50% and for Semester End Exam (SEE) is 50%. The minimum passing mark for the CIE is 40% of the maximum marks (20 marks). A student shall be deemed to have satisfied the academic requirements and earned the credits allotted to each course. The student has to secure not less than 35% (18 Marks out of 50) in the semester-end examination (SEE).

Continuous Internal Evaluation (CIE):

CIE marks for the practical course is **50 Marks**.

The split-up of CIE marks for record/ journal and test are in the ratio **60:40**.

- Each experiment to be evaluated for conduction with observation sheet and record write-up. Rubrics for the evaluation of the journal/write-up for hardware/software experiments designed by the faculty who is handling the laboratory session and is made known to students at the beginning of the practical session.
- Record should contain all the specified experiments in the syllabus and each experiment write-up will be evaluated for 10 marks.
- Total marks scored by the students are scaled downed to 30 marks (60% of maximum marks).
- Weightage to be given for neatness and submission of record/write-up on time.
- Department shall conduct 02 tests for 100 marks, the first test shall be conducted after the 8th week of the semester and the second test shall be conducted after the 14th week of the semester
- In each test, test write-up, conduction of experiment, acceptable result, and procedural knowledge will carry a weightage of 60% and the rest 40% for viva-voce.
- The suitable rubrics can be designed to evaluate each student's performance and learning ability. Rubrics suggested in Annexure-II of Regulation book
- The average of 02 tests is scaled down to **20 marks** (40% of the maximum marks).

The Sum of scaled-down marks scored in the report write-up/journal and average marks of two tests is the total CIE marks scored by the student.

Semester End Evaluation (SEE):

SEE marks for the practical course is 50 Marks.

SEE shall be conducted jointly by the two examiners of the same institute, examiners are appointed by the University

All laboratory experiments are to be included for practical examination.

VTUSYNC.IN

(Rubrics) Breakup of marks and the instructions printed on the cover page of the answer script to be strictly adhered to by the examiners. OR based on the course requirement evaluation rubrics shall be decided jointly by examiners.

Students can pick one question (experiment) from the questions lot prepared by the internal /external examiners jointly.

Evaluation of test write-up/ conduction procedure and result/viva will be conducted jointly by examiners.

General rubrics suggested for SEE are mentioned here, writeup-20%, Conduction procedure and result in -60%, Viva-voce 20% of maximum marks. SEE for practical shall be evaluated for 100 marks and scored marks shall be scaled down to 50 marks (however, based on course type, rubrics shall be decided by the examiners).

Change of experiment is allowed only once and 15% Marks allotted to the procedure part to be made zero.

The duration of SEE is 03 hours.

Rubrics suggested in Annexure-II of Regulation book

VTUSYNC.IN

LIST OF EXPERIMENTS

PART A		
Sl no	**Name of experiment**	**Page no.**
	Sample Inverter Program	
1	4- Bit Adder	
2	4 bit shift and add multiplier	
3	32-Bit ALU Supporting 4-Logical and 4-Arithmetic operations,	
4	Flip-Flops (D, SR and JK)	
5	Four-bit Synchronous MOD-N counter with Asynchronous reset	
PART B		
6	Schematic and Layout of CMOS Inverter	
7	Schematic of 2-input CMOS NOR gate	
8	Schematic of the Boolean Expression Y= AB+CD+E using CMOS Logic	
9	Schematic and layout of Common source amplifier with PMOS current mirror load	
10	Schematic and layout of two-stage operational amplifier	

VTUSYNC.IN

PART - A
DGITAL DESIGN
STEPS FOR HANDLING THE XILLINX

I). Design Entry and Simulation:

1. Desktop ⇨ virtual machine icon
2. *Open------ project navigator*
3. Go to File ⟶ New project - ⟶Give name to project ⟶click Next
4. Enter the specification of the Target device as

   Sparten 3, XCS 400, TQ144/PQ244 and Speed grade as 4/5

   or **default selection**
5. Select Verilog language in same window⟶click **next**
6. Click on **NEW Source**⟶ click on Verilog module⟶ give name of the module
7. Click **next** 4 times and finally click **Finish**
8. Enter the program in Verilog Design editor⟶ click on **SAVE**
9. Select 'synthesize XST 'click on **Check syntax, if any error, Clear it**

VTUSYNC.IN

II) To synthesize design:

1. Select 'synthesize XST 'click on **view synthesis report** and observe the **'Device utilization summary',**
2. Click RTL schematic and view the diagram

III). To enter Testbench Program and to perform Simulation:

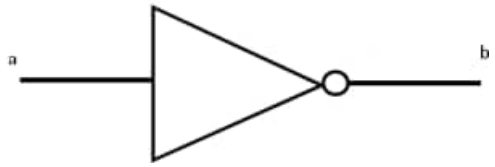
1. Click on **simulation** in Source Click on **'create new source**' on process
2. Click on 'Verilog **test fixture**' give name to test bench program → click **next**
3. Enter the test bench program → click on **SAVE**
4. click on **Check syntax**, if any error, Clear it
5. click on 'Simulate **behavioral** 'in process
6. Waveform will appear on simulation window

VTUSYNC.IN

Sample Program: INVERTER

AIM: To write Verilog code for inverter and to perform simulation, synthesis and study the area and timing constraints

Logic Diagram



Truth Table:

A (i/p)	B(o/p)
0	1
1	0

Verilog program for inverter:

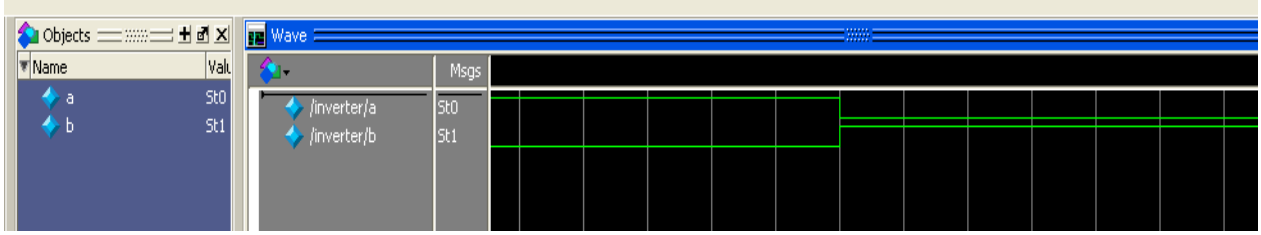
```
module inverter (a, b);

 input a;
output b;
assign b = ~a;
endmodule
```

Test Bench program:

```
module inverter_tb;
reg a;
wire b;
inverter uut (a, b); initial
begin

a=1'b0;
#100  a =1'b1;
#100 a=1'b0;
#100  a =1'b1;
#100;
end initial begin
$monitor ($time, " a=%b and
b=%b", a, b); end
endmodule
```

VTUSYNC.IN

SIMULATION RESULT OF INVERTER:



Synthesis Report:

Device Utilization for xcs400tq1444
****

Resource	Used	Avail	Utilization
IOs	1	232	1.29%
Global Buffers	0	24	0.00%
LUTs	1	9312	0.01%
CLB Slices	1	4656	0.02%
Dffs or Latches	0	9776	0.00%
Block RAMs	0	20	0.00%
Block Multipliers	0	20	0.00%

VTUSYNC.IN

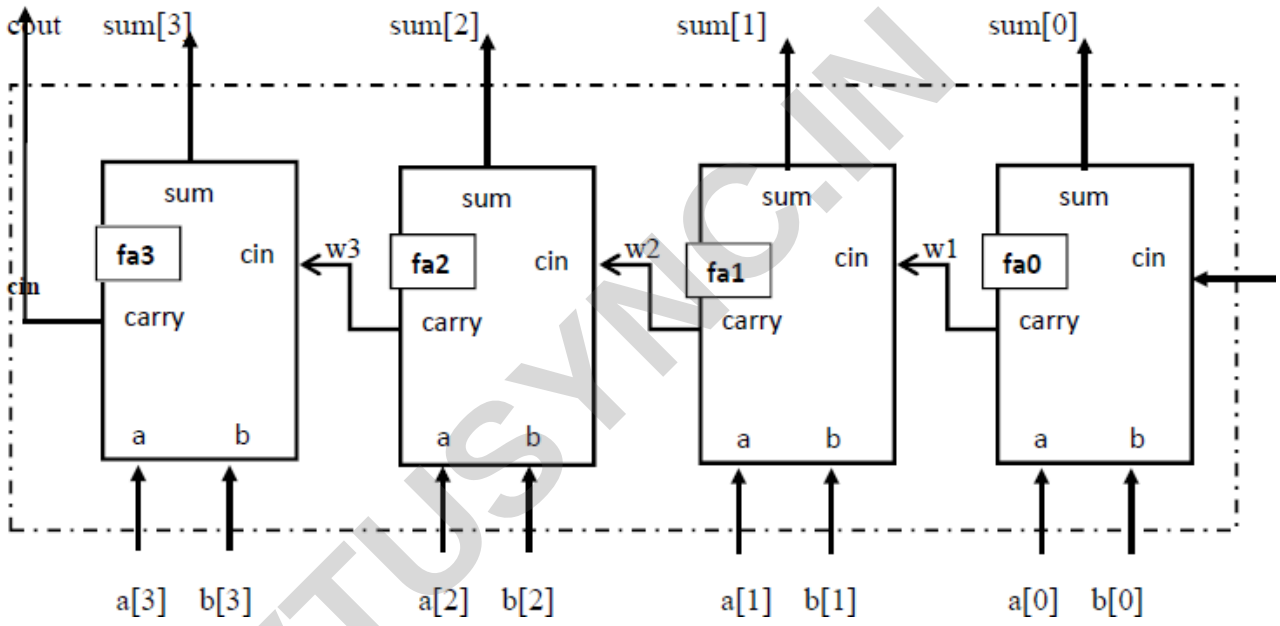
4-BIT ADDER

4 BIT PARELLEL ADDER:

Aim: To write a Verilog code for 4-Bit adder and verify the functionality using Test bench.

- Synthesize, Analyze Reports and Netlist, Critical Path and Max Operating Frequency.
- From the report generated, find the total number of cells, power requirement and total area requirement.

Block diagram:



VTUSYNC.IN

Truth Table:

a	b	c	s	C_{out}
0 0 11 0 0 0 1 1 1 0 0	1 1 0 0 0 1 1 1 0 1 1 1	0 1 0	1 1 1 1 1 0 0 0 0 0 1 1	0 0 1

Verilog Program:

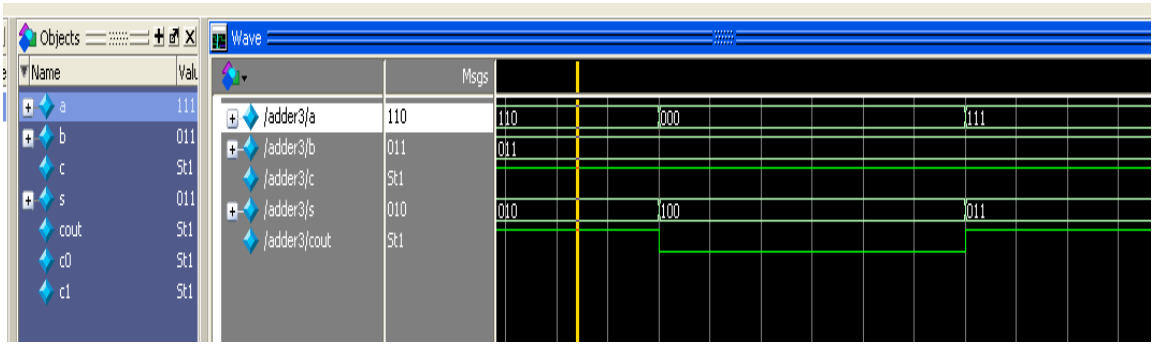
```
module parellel (a, b, cin, sum, carry);
input [3:0] a,b;
input cin;
output [3:0] sum;
```

```
output carry;
wire [3:1] w;

fulladder fa0(.a[0], b[0],. cin,.sum[0],.w[1]);
fulladder fa1(.a[1], b[1],. w[1],.sum[1],.w[2]);
fulladder fa2(.a[2], b[2],. w[2],.sum[2],.w[3]);
fulladder fa3(a[3],.b[3],.w[3],.sum[3],.carry);
endmodule
module fulladder (a, b, cin, sum, carry);
input a, b, cin;
output sum, carry;
assign sum=a ^ b^ cin;
assign carry= (a & b)| (b & cin) | (cin & a);
endmodule
```

Test Bench program:

```
module parellel_tb;
reg [3:0] a,b;
reg cin;
wire [3:0] sum;
wire carry;
parellel   uut (a, b, cin, sum, carry);
initial
begin
a=4'b0111; b=4'b0100; cin=1'b0;
#10; a=4'b1011; b=4'b0110; cin=1'b1;
end
initial
$monitor ($time, "a=%b b=%b cin=%b sum=%b carry=%b",a, b, cin, sum, carry);
Endmodule
```

Simulation result parallel adder:



VTUSYNC.IN

Synthesis report:

Device Utilization for 3S500EFG320

****

Resource	Used	Avail	Utilization
IOs	5	232	2.16%
Global Buffers	0	24	0.00%
LUTs	2	9312	0.02%
CLB Slices	1	4656	0.02%
Dffs or Latches	0	9776	0.00%
Block RAMs	0	20	0.00%
Block Multipliers	0	20	0.00%

Conclusion:

VTUSYNC.IN

4-bit shift and add multiplier

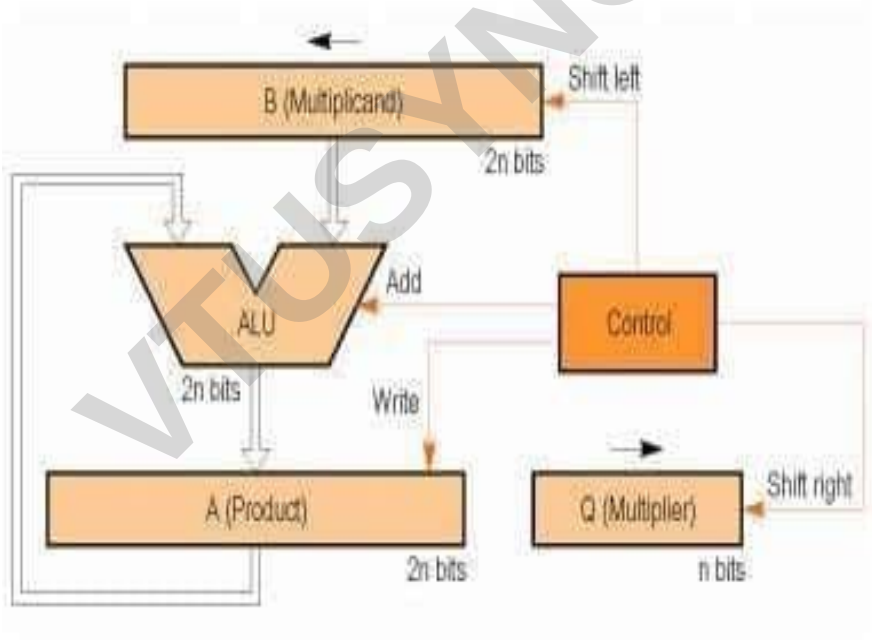
Aim: To write a Verilog code for 4-Bit Shift and Add Multiplier and verify the functionality using Test bench.

- Synthesize, Analyze Reports and Netlist, Critical Path and Max Operating Frequency.
- From the report generated, find the total number of cells, power requirement and total area requirement.

Design Information and Flow Chart:

Binary multipliers are used for multiplication of 2 binary numbers and are used mainly in signal processing and also in other computationally intensive applications. Shift and add binary multiplier is a type of sequential multiplier. Sequential multipliers generate the partial products sequentially and add each newly generated partial product to the previously accumulated sum. Shift and add binary multiplier is a type of sequential multiplier

First Version of the Multiplier Circuit



First version of the multiplier circuit.

VTUSYNC.IN

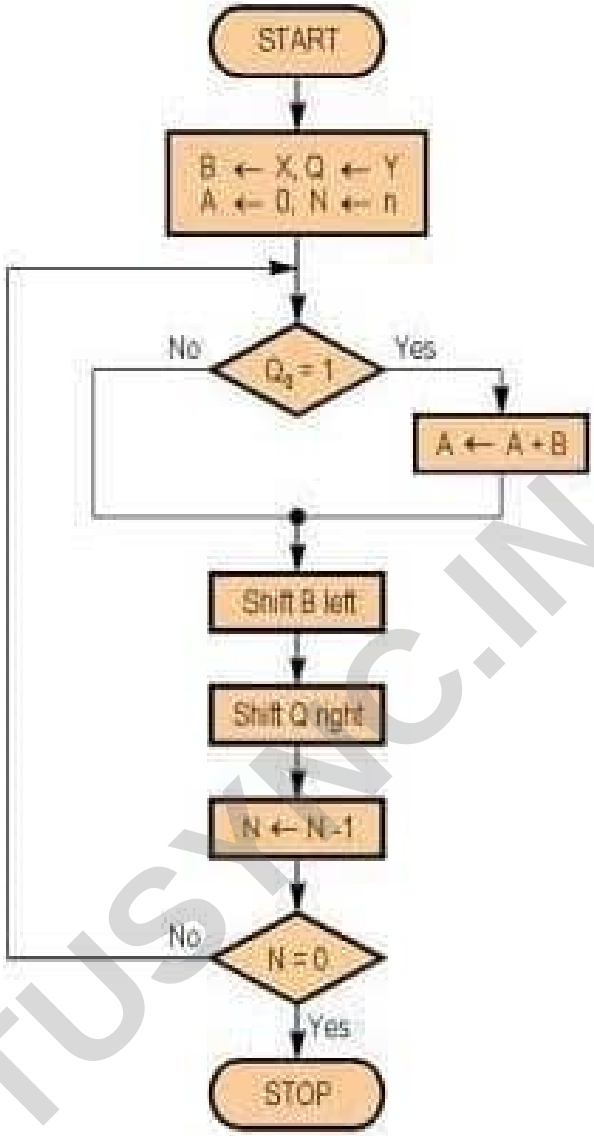


Figure No 2.2: Flow Chart of Shift and Add Multiplier Algorithm

Shift and add multiplier is similar to multiplication done by paper and pencil. This method adds the multiplicand X to itself Y times, where Y denotes the multiplier. In the case of binary multiplication, since the digits are 0 and 1, if the multiplier digit is 1, a copy of the multiplicand is placed in the proper positions; if the multiplier digit is 0, a number of 0 digits are placed. The 2n- bit product register (A) is initialized to 0. A 2n-bit multiplicand register with the multiplicand placed in the right half of the register and with 0 in the left half is used. The algorithm starts by loading the multiplicand into the B register, loading the multiplier into the Q register, and initializing the A register to 0. The counter N is initialized to n. The least significant bit of the multiplier register (Q0) determines whether the multiplicand is added to the product register. The left shift of the multiplicand has the effect of shifting the intermediate products to the left and right shift prepares the next bit of the multiplier to examine in the following iteration.

VTUSYNC.IN

Verilog Code:

```
`timescale 1ns/1ns
module shift_and_add_binary_multiplier( clk,rst,A, B, C);
parameter m=4, n=4;
integer i;
input clk,rst;
input [m-1:0] A;
input [n-1:0] B;
output reg [m+n-1:0]C;
reg [m+n-1:0]A1;
reg [n-1:0]B1;
always@(posedge clk or posedge rst )
begin
                if (rst)
                begin
                C=0;
                end
                else
                begin
                C=0;
                A1[m-1:0]=A;
                A1[m+n-1:m]=0;
                B1=B;
        for (i=0;i<n;i=i+1)
        begin
                        if(B1[i]==1'b0)
                        begin
                                C=C+0;
                        end
                        else if (B1[i]==1'b1)
                        begin
                                C=C+(A1<<i);
                        end
                end
                end
end
endmodule
```

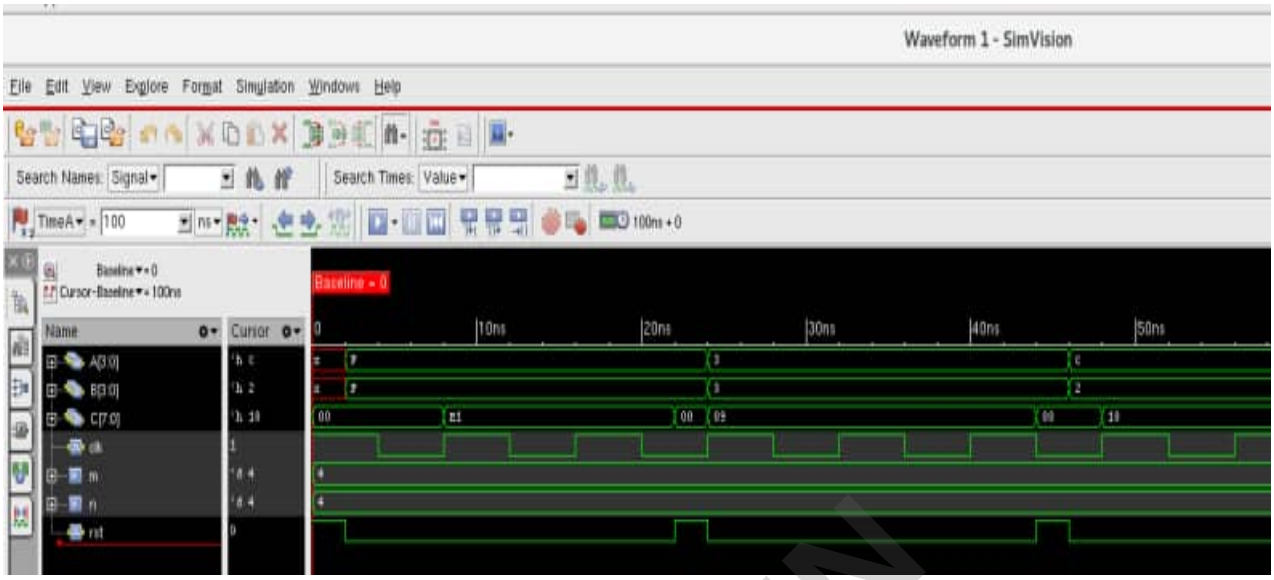
VTUSYNC.IN

Testbench Program:

```
`timescale 1ns/1ns
module shift_and_add_binary_multiplier_tb;
parameter m=4, n=4;
reg clk, rst;
reg [m-1:0] A;
reg [n-1:0] B;
wire [m+n-1:0] C;

shift_and_add_binary_multiplier uut(clk, rst, A, B, C);

initial
begin
    clk = 1'b1;
    forever #4 clk = ~clk;
end
initial
begin
    rst = 1;
 #2 rst = 0;
    A = 4'b1111;
    B = 4'b1111;
 #20 rst = 1;
 #2  rst = 0;
    A = 4'b0011;
    B = 4'b0011;
 #20 rst = 1;
 #2  rst = 0;
    A = 4'b1100;
    B = 4'b0010;
 #20;
end

initial
begin #100 $finish;
end

endmodule
```

VTUSYNC.IN

Waveforms:



Synthesis Report:

Conclusions:

VTUSYNC.IN

32 bit ALU

Aim: 32-Bit ALU Supporting 4-Logical and 4-Arithmetic operations, using case and if statement for ALU Behavioral Modeling

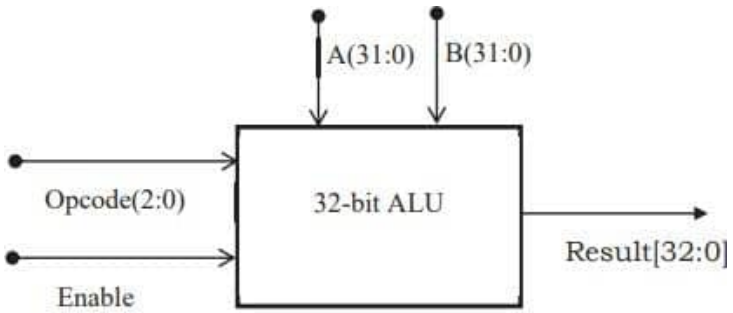


Figure 1 ALU top level block diagram

Table 1 ALU Functions

Opcode (3:0)	ALU Operation	Remarks
0000	A + B	Addition of two numbers
0001	A – B	Subtraction of two numbers
0010	A & B	// Bitwise AND
0011	A OR B	Bitwise OR
0100	A XOR B	Bitwise EX OR
0101	~(A&B)	Bitwise NAND
0110	A<<B	Left shift operand_A by operand_B[4:0] bits
0111	A>>B	Right shift operand_A by operand_B[4:0] bits
0111	A+1	Increment A by 1
1000	A-1	Decrement by 1
1001	~A	Complement A

VTUSYNC.IN

Verilog code:

```
module ALU32 (input en, input [0:3] op_code , input [31:0] A,
                input [31:0] B, output reg ack, output reg [32:0] result);

always@(en,op_code,A,B)
begin
        if(en)
 case(op_code)
4'b0000: res= A+B;
4'b0001: res= A-B;
4'b0010: res= A&B;
4'b0011: res= A|B;
4'b0100: res= A^B;
 4'b0101: res= A&B;
4'b0110: res= A<<B;
4'b0111: res= A>>B;
4'b1000: res= A+1;
4'b1001: res= A-1;
4'b1010: res= ~A;
 default: res=33'hxxxxxxxx;
endcase
else
res= 33'hzzzzzzzz;
ack = 1'b1;
 #5 ack =1'b0;
end
endmodule
```

Test Bench:

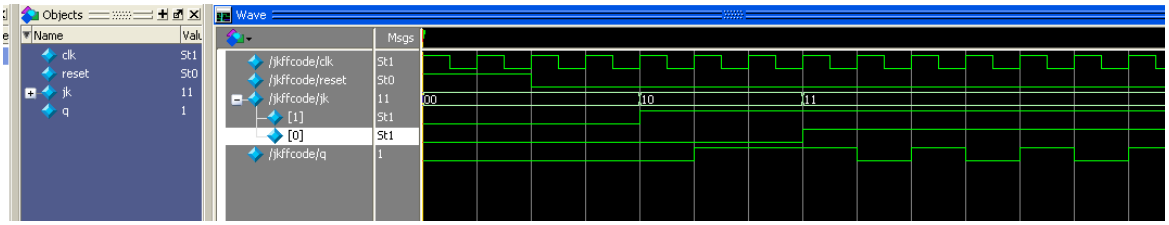
```
module TB_ALU32;
        reg en;
        reg [3:0] op;
        reg [31:0] A;
        reg [31:0] B;

        wire [32:0] res;
        wire ack;
```

VTUSYNC.IN

```
// Instantiate the Unit Under Test (UUT)
ALU32 uut (
        .en(en),
        .op(op),
        .A(A),
        .B(B),
        .ack(ack),
        .res(res) );
initial
        begin
        op = 02;
        A = 20;
        B = 10;

        // Wait 50 ns for global reset to finish#50;

        // Add stimulus here

        en = 1;
        A = 32'hffff_ffff;
        B = 32'h2;
        op = 0;
        #50: op=1;
        #50: op=2;
        #50: op=3;
        #50: op=4;
        #50: op=5;
        #50: op=6;
        #50: op=7;
        #50: op=3'b1xx;
        #50;
end
endmodule
```

VTUSYNC.IN

Result obtained:



Simulation Waveform of 32-Bit ALU

Synthesis Result of ALU:

Device Utilization for 3S400tq1444
****

Resource	Used	Avail	Utilization
IOs	4	232	1.72%
Global Buffers	1	24	4.17%
LUTs	2	9312	0.02%
CLB Slices	1	4656	0.02%
Dffs or Latches	1	9776	0.01%
Block RAMs	0	20	0.00%
Block Multipliers	0	20	0.00%

32 bit ALU using if statement

```
module alu_32bit_if(y,a,b,f);
input [31:0]a;
input [31:0]b;
input [2:0]f;
output reg [31:0]y;
always@(*)
begin
if(f==3'b000)
y=a&b;                    //AND Operation
else if (f==3'b001)
y=a|b;                    //OR Operation
else if (f==3'b010)
y=a+b;                    //Addition
else if (f==3'b011)
y=a-b;                    //Subtraction
else if (f==3'b100)
y=a*b;                    //Multiply
else
y=32'bx;
end
endmodule
```

VTUSYNC.IN

Test bench:

```
module alu_32bit_tb_if; reg
[31:0]a;
reg [31:0]b;
reg [2:0]f;
wire [31:0]y;
alu_32bit_if test(.y(y),.a(a),.b(b),.f(f)); initial
begin
a=32'h00000000;
b=32'hFFFFFFFF; #10
f=3'b000;
#10 f=3'b001;
#10 f=3'b010;
#10 f=3'b100;
end
initial
#50 $finish;
endmodule
```

Wave Forms :

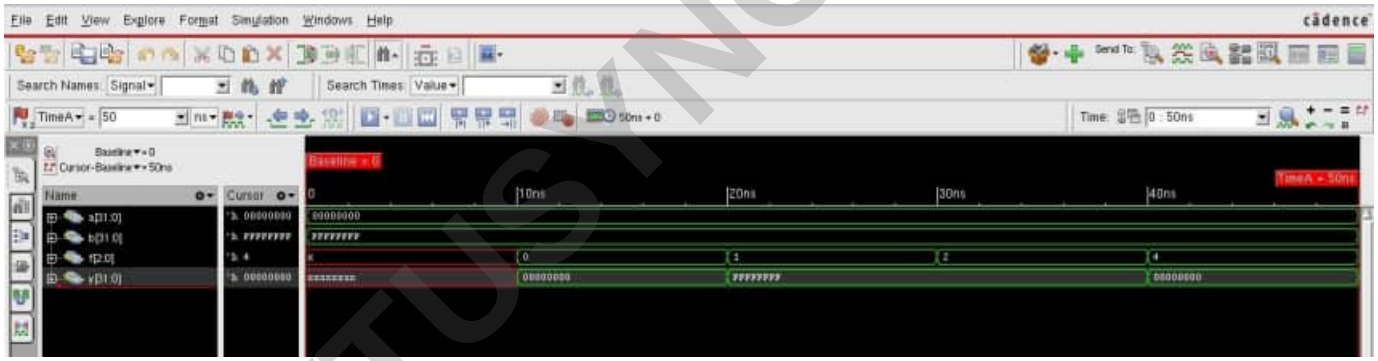


Figure No 3.2: Simulation Waveform of 32-Bit ALU

Conclusion:

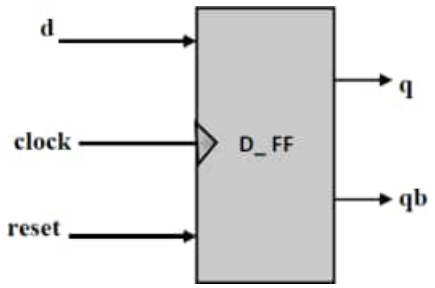
VTUSYNC.IN

FLIP-FLOPS

AIM: To write verilog code for all the types of flip-flops and to perform simulation , synthesis and study the area and timing constraints

D flip flops :

Block diagram



Truth Table

CLK	RST	Data	Q
↑	1	x	0
↑	0	0	0
↑	0	1	1

Verilog code

```
module dff (reset, clock, d, q,
qb); input reset, clock, d;
output q,
qb; reg q;
wire qb;
always @ (posedge clock)
begin
if
(reset)
q<=1'b
0;
else
q<=d;
end
assign
qb=~q;
endmodule
```

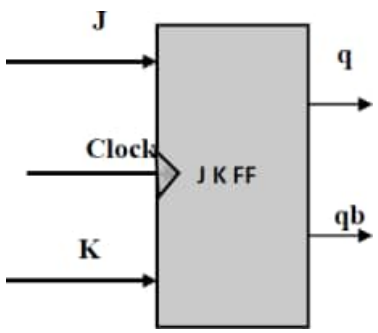
Test bench program

```
module  dff_tb;
reg clock, reset,
d; wire q, qb;
dff uut (reset, clock, d, q,
qb); initial
begin
clock=1'b1;
reset=1'b0;
end
always #5 clock=~clock;
always #40 reset=~reset;
initial  begin
#20 d =1'b1;
#20 d =1'b0;
#30 d =1'b1;
#30 d =1'b0;
End
initial
begin
$monitor ($time, "reset=%b clock=%b d=%b q=%b
qb=%b", reset, clock, d, q, qb);
end
endmodule
```

VTUSYNC.IN

JK Flip Flop:

Block diagram



Truth Table

CLK	RESET	JK	Q
☐	1	XX	0
☐	0	00	Q
☐	0	01	0
☐	0	10	1
☐	0	11	qbar .(Toggle)

Verilog code

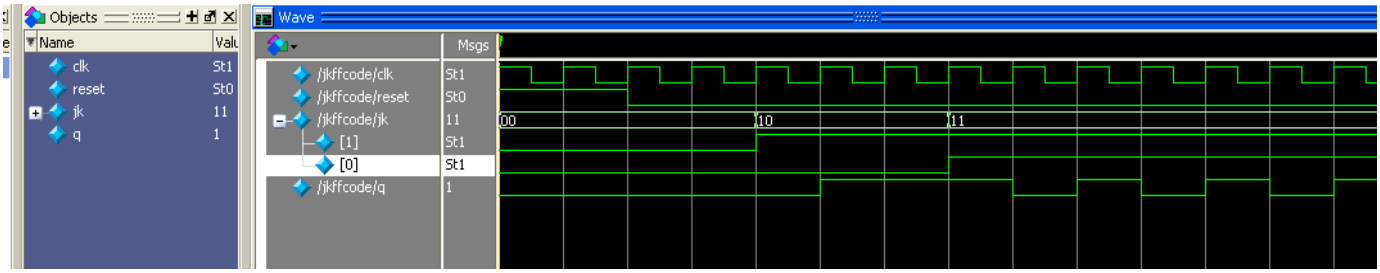
```
module JK_FF (JK, clock, q, qb);
input [1:0] JK;
input clock;
output reg q, qb;
always @ (posedge clock)
begin
case (JK)
2'b00 : q = q;
2'b01 : q = 1'b0;
2'b10 : q = 1'b1;
2'b11 : q =~ q;
endcase
qb =~ q;
end
endmodule
```

Test bench program :

```
module JK_ff_tb;
reg [1:0] JK;
reg clock;
wire q, qb;
JK_FF uut(JK, clock, q, qb);
initial
clock=1'b1;
always #5 clock=~clock;
initial
begin
JK=2'b00; #20;
JK=2'b01; #20;
JK=2'b10; #20;
JK=2'b11; #50;
end
initial
$monitor ($time, "JK=%b q=%b qb=%b ", JK, q, qb);
endmodule
```

VTUSYNC.IN

Simulation Result of JK flip flop:



Synthesis Result of JK flip flop :

Device Utilization for 3S400tq1444

****

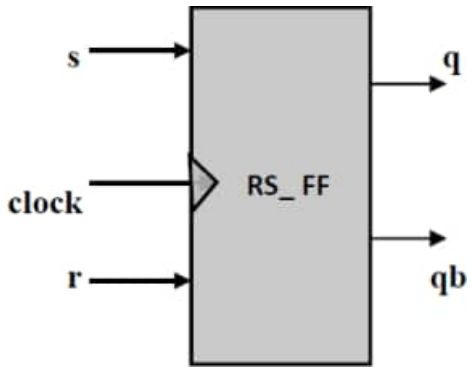
Resource	Used	Avail	Utilization
IOs	4	232	1.72%
Global Buffers	1	24	4.17%
LUTs	2	9312	0.02%
CLB Slices	1	4656	0.02%
Dffs or Latches	1	9776	0.01%
Block RAMs	0	20	0.00%
Block Multipliers	0	20	0.00%

Conclusion:

VTUSYNC.IN

RS flip flops:

Block diagram



Truth Table

CLK	Reset	S R	Q
	1	X	0
	0	X	Q
	0	00	0
	0	01	1
	0	10	Indetermitant
		11	

Verilog code

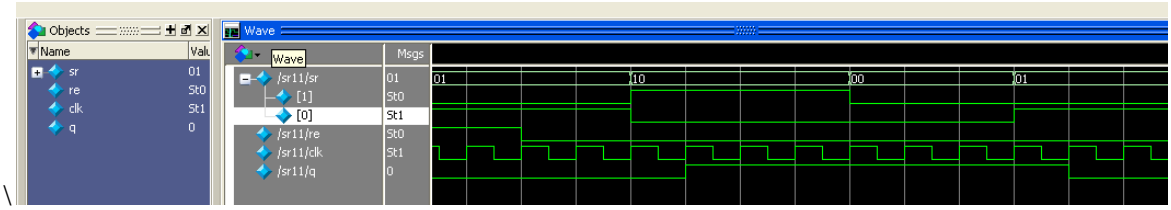
```
module rs_ff(rs, clock, q, qb);
input [1:0] rs;
input clock;
output reg q, qb;
always @ (posedge clock)
begin
case (rs)
2'b00 : q = q ;
2'b01 : q = 1'b1 ;
2'b10 : q = 1'b0 ;
2'b11 : q = 1'dZ ;
endcase
qb =~ q;
end
endmodule
```

Test bench program

```
module rs_ff_tb;
reg [1:0] sr;
reg clock;
wire q, qb;
rs_ff uut(rs, clock, q, qb);
initial
clock=1'b1;
always #5 clock=~clock;
initial
begin
rs=2'b00; #20;
rs =2'b01; #20;
rs =2'b10; #20;
rs =2'b11; #50;
end
initial
$monitor ($time, "rs=%b q=%b qb=%b ", rs, q, qb);
endmodule
```

VTUSYNC.IN

Simulation Result of SR flip flop:



Synthesis Result of SR flip flop:

Device Utilization for 3S500EFG320

****

Resource	Used	Avail	Utilization
IOs	4	232	1.72%
Global Buffers	1	24	4.17%
LUTs	2	9312	0.02%
CLB Slices	1	4656	0.02%
Dffs or Latches	1	9776	0.01%
Block RAMs	0	20	0.00%
Block Multipliers	0	20	0.00%

Conclusion:

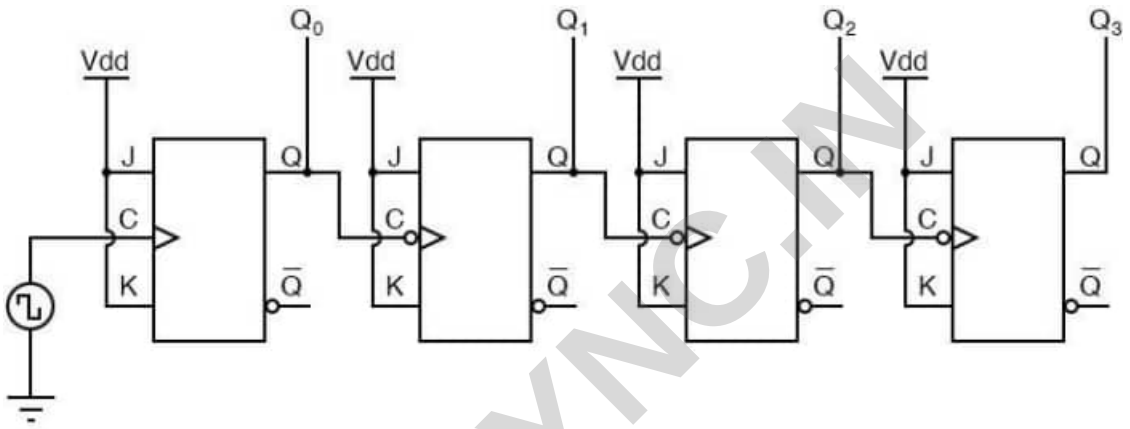
VTUSYNC.IN

4 BIT UP/DOWN ASYNCRONUS COUNTER

Aim: Write Verilog code for 4-bit up/down asynchronous reset counter and carry out the following:

a. Verify the functionality using test bench
b. Synthesize the design by setting area and timing constraint. Obtain the gate level netlist,

<u>Asyncronus counter Diagram:</u>



Truth Table:

Clk	rst	Count
□	1	0 0 0 0
□	0	Counting from 0001 to 1111 for every clock

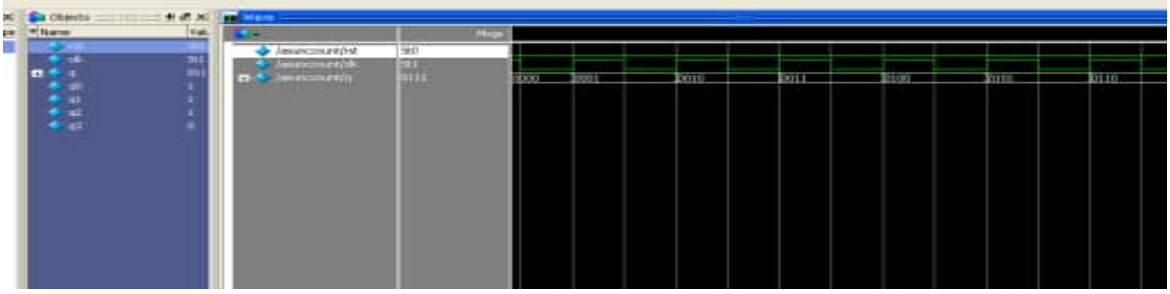
VTUSYNC.IN

Verilog Programme	Testbench program

module asynccount(clk,rst,q) ;
input rst,clk;
output [3:0]q;
reg q0,q1,q2,q3;
always @(posedge clk or posedge rst)
begin
if(rst)
q0=1'b0;
else
q0=~q0;
end
always @(negedge q0 or posedge rst)
begin
if(rst)
q1=1'b0;
else
q1=~q1;
end
always @(negedge q1 or posedge rst)
begin
if(rst)
q2=1'b0;
else
q2=~q2;
end
always @(negedge q2 or posedge rst)
begin
if(rst)
q3=1'b0;
else
q3=~q3;
end
assign q={q3,q2,q1,q0};
Endmodule
	module asyncounter_tb;
	reg clock, reset;
	wire [3:0] q;
	asyncounter uut(clock, reset, q);
	initial
	clock=1'b0;
	always #10 clock=~clock;
	initial
	begin
	#20; reset=1'b1;
	#20; reset=1'b0;
	end
	initial
	$monitor ($time, " reset=%b q = %b" ,reset, q);
	Endmodule

VTUSYNC.IN

SIMULATION RESULT OF ASYNCRONUS COUNTER:



Synthesis result of synchronous counter:

Device Utilization for XCS400tq144
****

Resource	Used	Avail	Utilization
IOs	5	66	7.58%
Global Buffers	1	24	4.17%
LUTs	4	1920	0.21%
CLB Slices	2	960	0.21%
Dffs or Latches	4	2052	0.19%
Block RAMs	0	4	0.00%
Block Multipliers	0	4	0.00%

****

Conclusion:

.

VTUSYNC.IN

MICROWIND TOOL

MICROWIND is truly integrated EDA software encompassing IC designs from concept to completion, enabling chip designers to design beyond their imagination. MICROWIND integrates traditionally separated front-end and back-end chip design into one flow, accelerating the design cycle and reduces design complexities.

It tightly integrates mixed-signal implementation with digital implementation, circuit simulation, transistor-level extraction and verification – providing an innovative education initiative to help individuals to develop the skills needed for design positions in virtually every domain of IC industry.

The MICROWIND software allows the designer to simulate and design an integrated circuit at physical description level. Born in Toulouse (France), Microwind is an innovative CMOS design tool for educational market.

Microwind is developed as comprehensive package on windows platform to enable students to learn smart design methods and techniques with more practice. With inbuilt layout editing tools, mix-signal simulator, MOS characteristic viewer and more, it allows students to learn complete design process with ease.

Microwind unifies schematic entry, pattern based simulator, SPICE extraction of schematic, Verilog extractor, layout compilation, on layout mix-signal circuit simulation, cross sectional & 3D viewer, netlist extraction, BSIM4 tutorial on MOS devices and sign-off correlation to deliver unmatched design performance and productivity.

With its approach for CMOS design education, Microwind has gained lot followers worldwide. Universities across the globe are using Microwind for budding engineers to teach CMOS concepts with ease. Paving their path for more skilled softwares to be used at later stage of their course work.

VTUSYNC.IN

PART II ANALOG DESIGN

Procedure for schematic design by using DSCH 1.
Open DSCH 3.1.

2. File tab -generate spice file.
3. To set path for spice file, select option for Browse- select the drive.
4. Construct the schematic circuit by dragging the components from the pallet.
5. Save and run.
6. Go to program files.
7. Select Microwind 2 folder.
7. Select client folder-select DSCH3.1 folder-select system folder.
8. Select spice library file-close the Window-Enable the ADD node list.
9. Check on update spice file.
10. Change to n.
11. Save the spice file and click ok.

Procedure for layout design by using Microwind

1. Open Microwind.
2. Open file tab-Open select foundry-select CMOS 0.12μm technology.
3. Construct layout by picking the layers from the pallet.
4. Save and simulate the layout.
5. Check DC and Transient analysis.
6. Calculate Gain of layout and verify with the practical result.
7. Extract RC values.
8. Note all the results- save and quit.

VTUSYNC.IN

CMOS INVERTER

Question:

I) **Capture the schematic of CMOS inverter with load capacitance of 0.1pF and set the widths of inverter with Wn=Wp, Wn=2Wp, Wn=Wp/2 and length at selected technology. Carry out the following:**

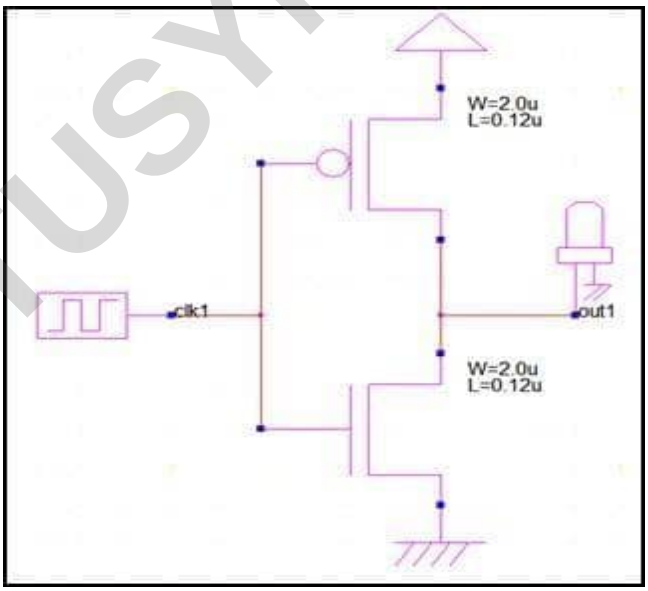
a. **Set the input signal to a pulse with rise time, fall time of 1ns and pulse width of 10ns and time period of 20ns and plot the input voltage and output voltage of designed inverter?**

b. **From the simulation results compute tp_{HL}, tp_{LH} and td for all three geometrical settings of width?**

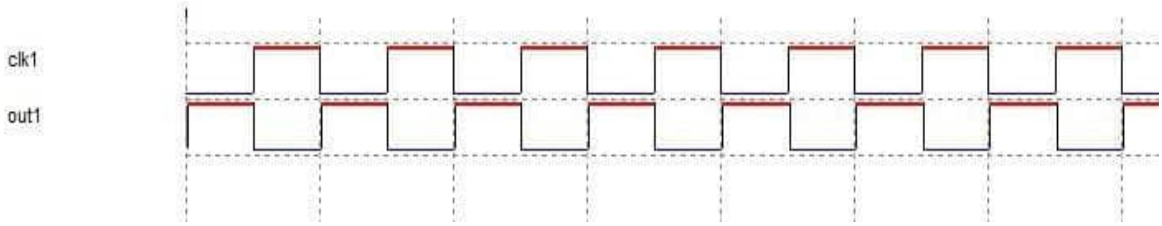
c. **Tabulate the results of delay and find the best geometry for minimum delay for CMOS inverter?**

Schematic design:

For NMOS and PMOS Wn=Wp, Wn=2Wp, Wn=Wp/2



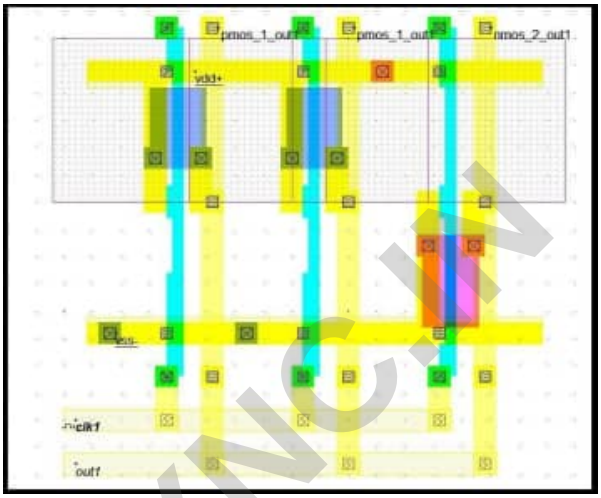
<u>Output waveform</u>



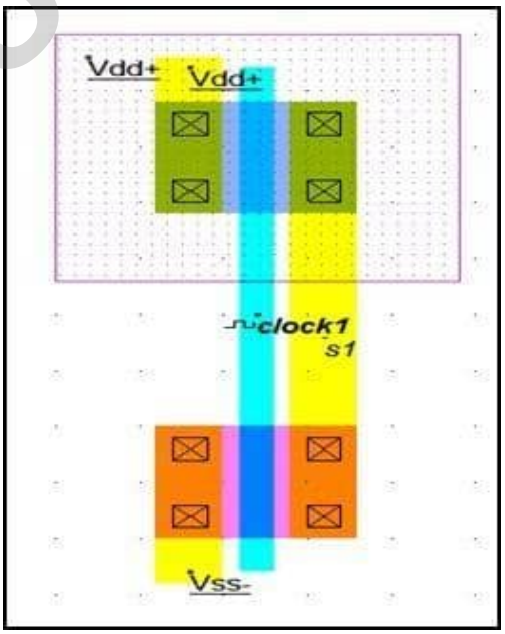
VTUSYNC.IN

Steps to convert the schematic to layout

1. Go to microwind2 and draw the schematic
2. Save the file with file name
3. Simulate the schematic for correct output
4. Go to file ------ Make a verilog file
5. Go to microwind 2
6. Go to compile
7. Compile verilog file created in the microwind 2
8. Layout of the schematic will be generated

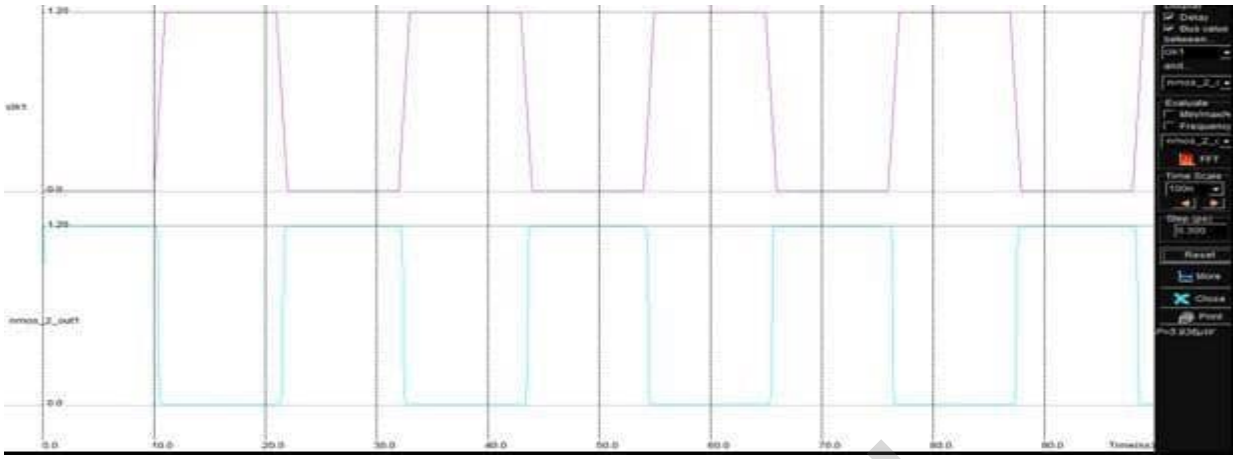


II). Draw layout of inverter with Wp/Wn = 40/20, use optimum layout methods. Verify for DRC and LVS,extract parasitic and perform post layout simulations, compare the results with pre-layout simulations. Record the observations.

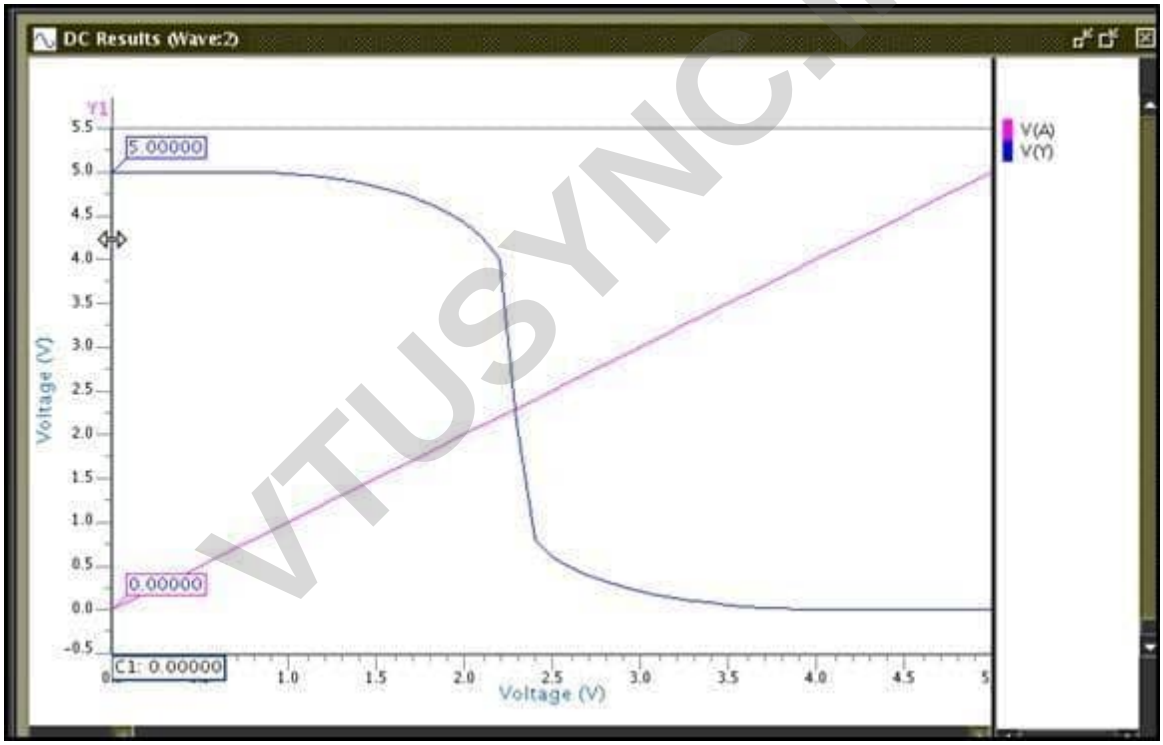


VTUSYNC.IN

Transient Chacterstics



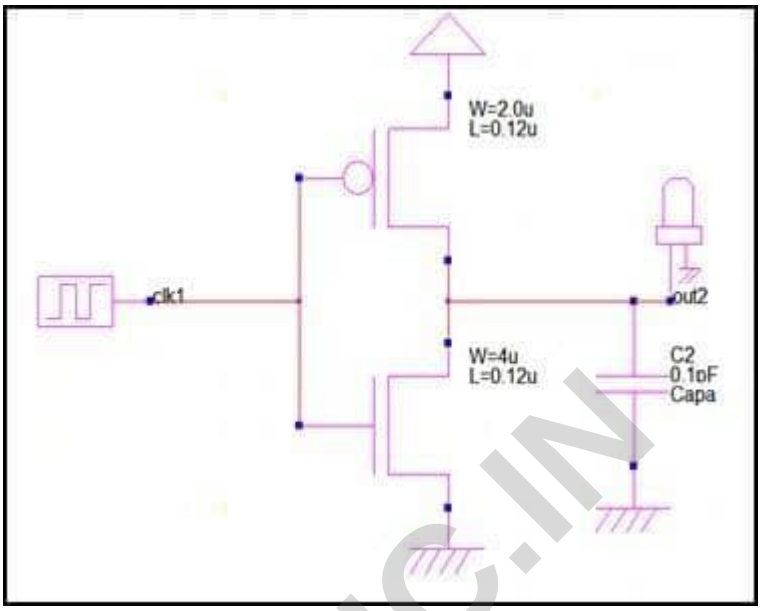
DC Characterstics



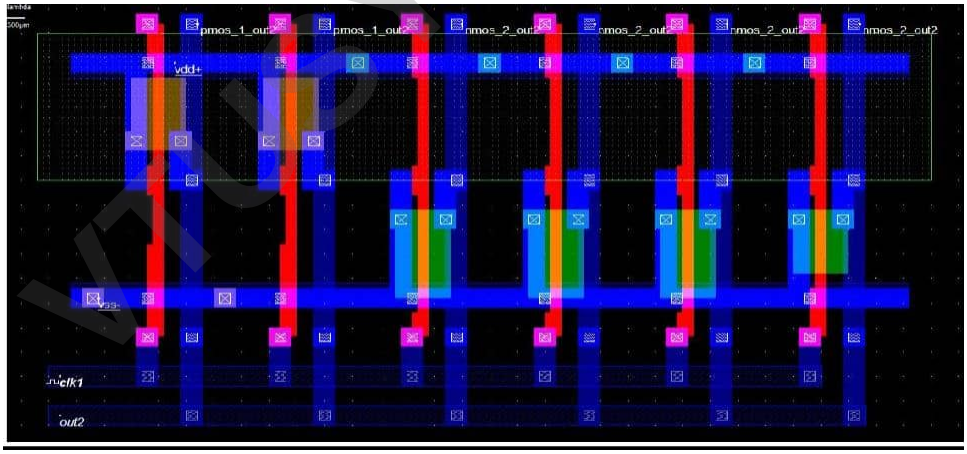
VTUSYNC.IN

For Wn=2Wp

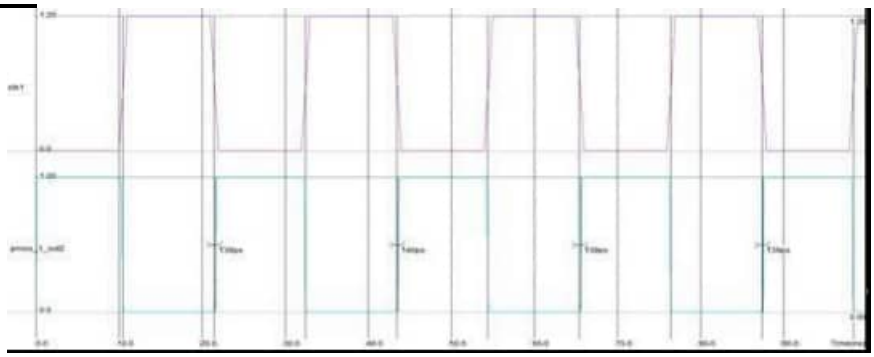
Schematic diagram



Generated layout for the verilog file



Output waveform



VTUSYNC.IN

Results:

SL No	Transistor geometry	Tplh	Tphl	Td
1	Wn=Wp,			
2	Wn=2Wp			
3	Wn=Wp/2			

Conclusion:

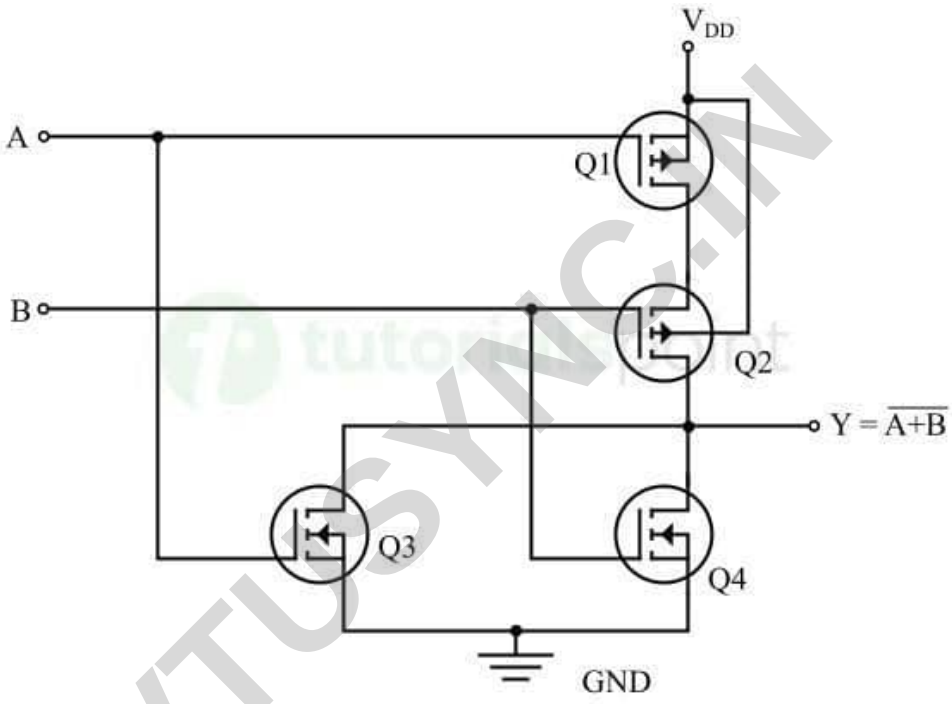
VTUSYNC.IN

SCHEMATIC OF 2-INPUT CMOS NOR GATE

AIM: Capture the schematic of 2-input CMOS NOR gate having similar delay as that of CMOS inverter computed in experiment above. Verify the functionality of NOR gate and also find out the delay td for all four possible combinations of input vectors

THEORY:

Similar to CMOS NAND gate, we can also design a NOR gate using PMOS and NMOS transistors. The circuit diagram of a two input NOR gate using CMOS technology is shown in the following figure.



VTUSYNC.IN

Case 1: When Input A is Low and Input B is Low

In this case, both PMOS transistors Q1 and Q2 will be ON and both NMOS transistors Q3 and Q4 will be OFF.

Under this switching condition of the CMOS transistors, there is a path between the supply voltage V_{DD} and the output terminal Y through the ON PMOS transistors. But there is no path between the output terminal Y and the ground terminal GND. This maintains the output at the voltage level V_{DD} and hence the output will be High.

Thus, when A = 0 and B = 0, then Y = 1

Case 2: When Input A is Low and Input B is High

In this case, the PMOS transistor Q1 is ON, the PMOS transistor Q2 is OFF, the NMOS transistor Q3 is OFF, and the NMOS transistor Q4 is ON.

Since, the PMOS transistors Q1 and Q2 are connected in series and the transistor Q2 is OFF. Thus, there is no path between the power supply V_{DD} and the output terminal Y. But there is a connection between the output line Y and the ground terminal GND through the ON NMOS transistor Q4. This sets the output terminal to ground voltage and makes the output Low.

Therefore, when A = 0 and B = 1, then Y = 0

Case 3: When Input A is High and Input B is Low

In this condition, the PMOS transistor Q1 is OFF, the PMOS transistor Q2 is ON, the NMOS transistor Q3 is ON, and the NMOS transistor Q4 is OFF.

In this case, there is no closed path between the power supply V_{DD} and the output line Y due to OFF PMOS transistor Q1. But there is a closed between the output line Y and the ground terminal GND through the ON NMOS transistor Q3. Hence, the output terminal is connected to the ground potential and makes the output Low.

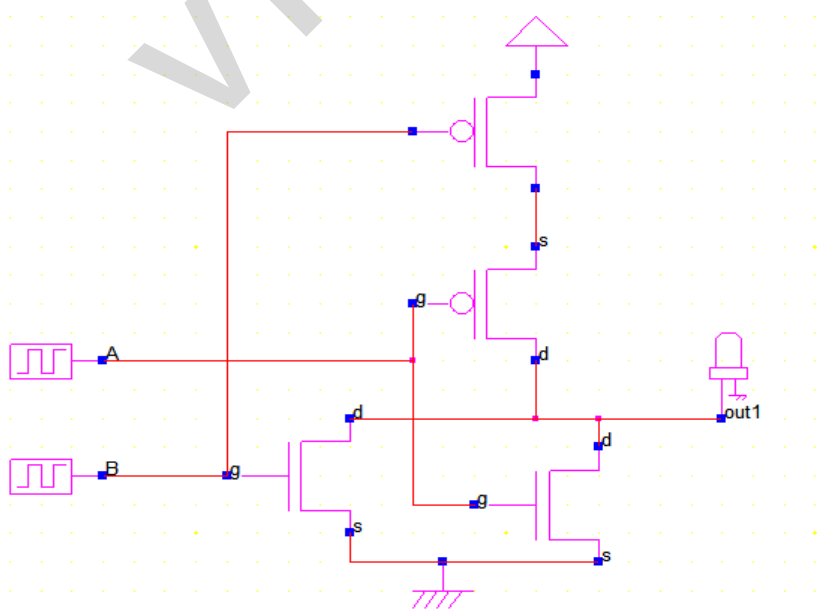
Thus, when A = 1 and B = 0, then Y = 0

Case 4 – When Input A is High and Input B is High

In this case, both PMOS transistors Q1 and Q2 are OFF and both NMOS transistors Q3 and Q4 are ON. Under this condition, there is no path between the supply voltage V_{DD} and the output terminal Y. But there is a closed path between the output terminal Y and the ground terminal GND. This maintains the output line at ground voltage level and hence the output will be Low.

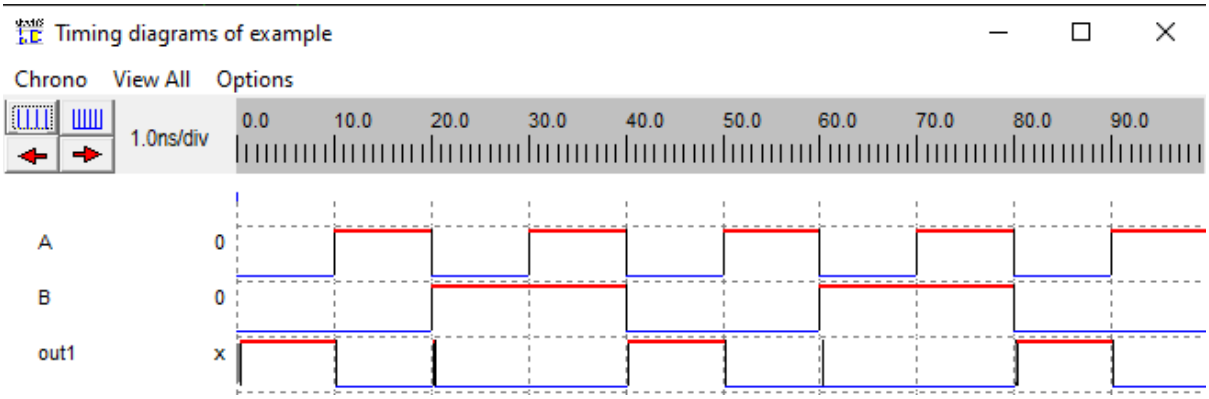
Thus, when A = 1 and B = 1, then Y = 0

Schematic diagram:



VTUSYNC.IN

Waveforms:



Measure delay between input and output:

Conclusion:

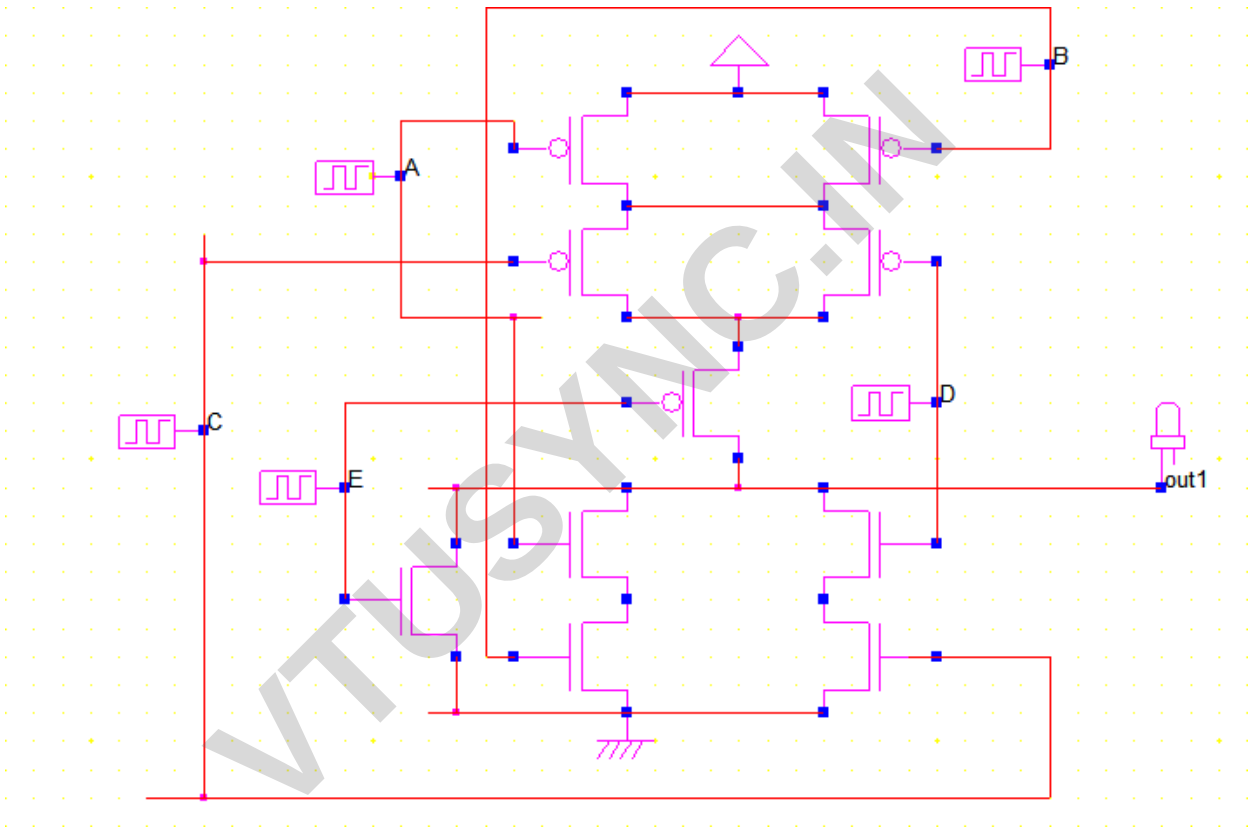
VTUSYNC.IN

Schematic of The Boolean Expression Using CMOS Logic

Aim: Construct the schematic of the Boolean Expression Y= AB+CD+E using CMOS Logic. Verify the functionality of the expression find out the delay td for some combination of input vectors.

Schematic design:

Boolean expression is $Y=\overline{AB+CD+E}$

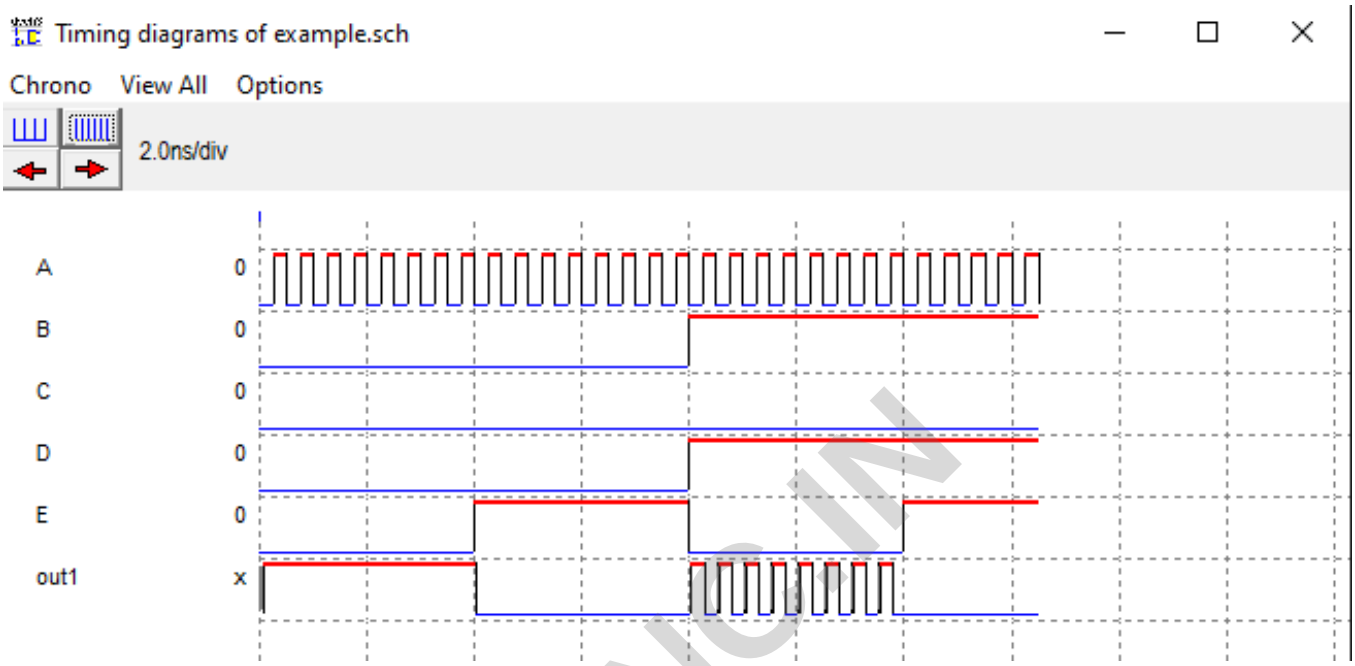


Truth table:

input ABCDE	Output Y $Y=\overline{AB+CD+E}$
00000	0
00001	1
00111	1
10000	0

VTUSYNC.IN

Waveform:



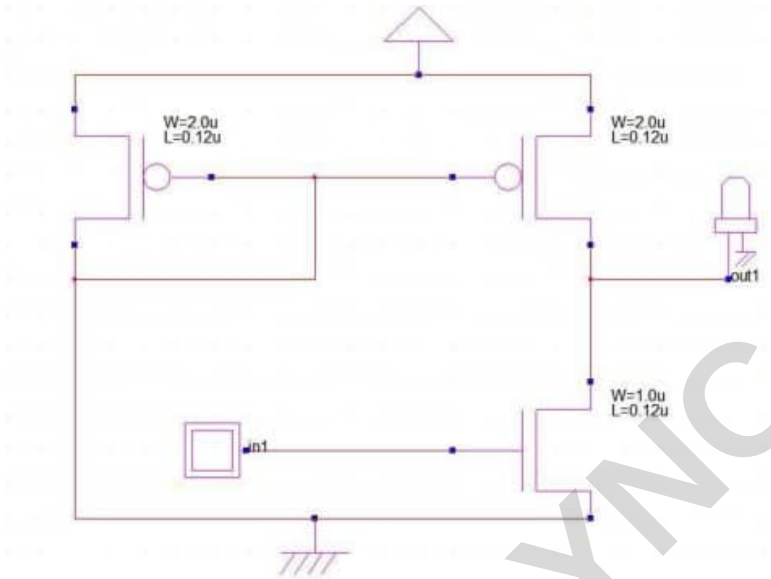
conclusion:

VTUSYNC.IN

COMMON SOURCE AMPLIFIER

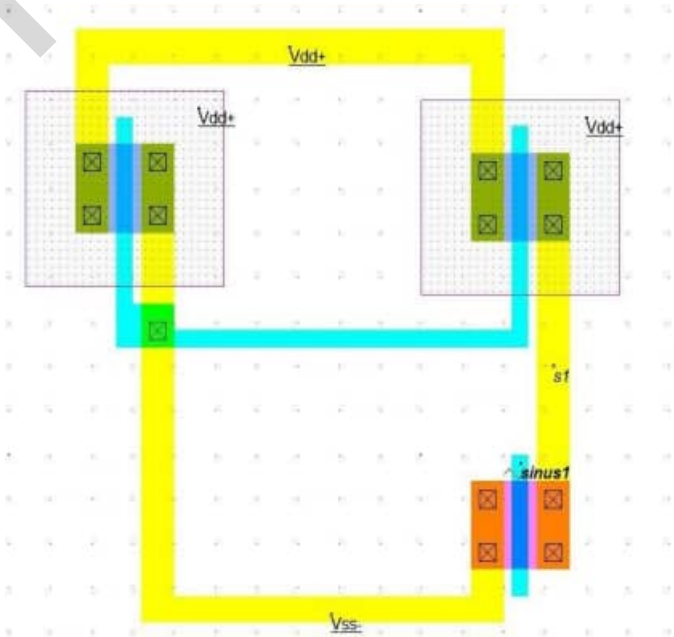
I).Capture schematic of Common Source Amplifier with PMOS Current Mirror Load and find its transient response and AC response? Measures the Unity Gain Bandwidth (UGB)

Schematic diagram



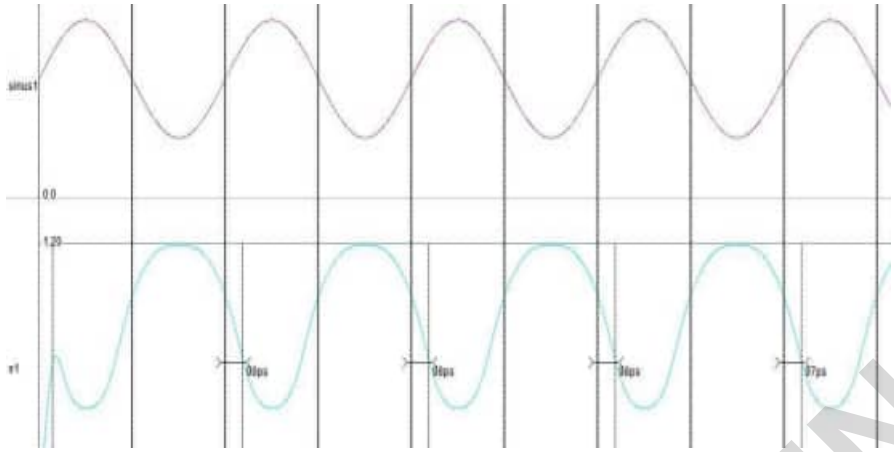
Draw layout of common source amplifier, use optimum layout methods. Verify for DRC and LVS, extract parasitic and perform post layout simulations, compare the results with pre-layout simulations. Record the observations.

Layout diagram

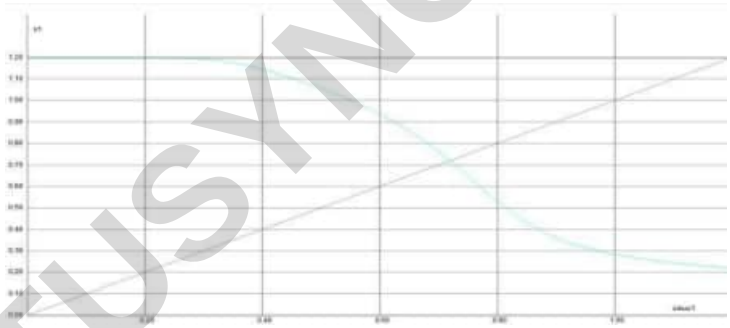


VTUSYNC.IN

Output waveform (voltage vs time characteristics)



Output waveform(Voltage vs voltage characteristics)



VTUSYNC.IN

Results:

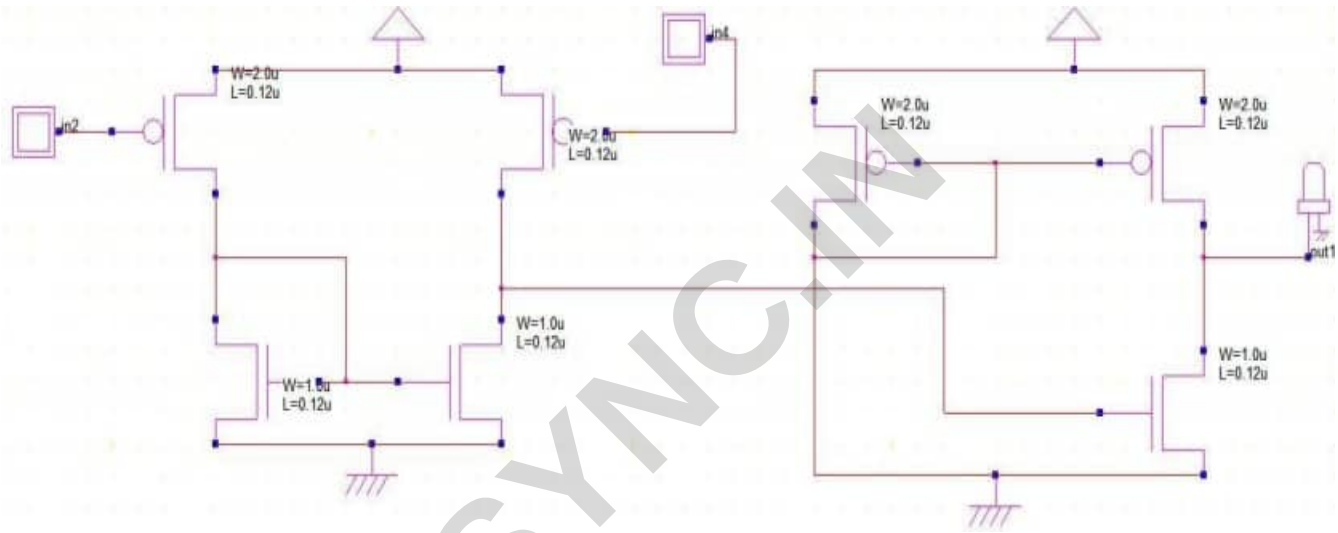
Conclusion:

Two-Stage Operational Amplifier

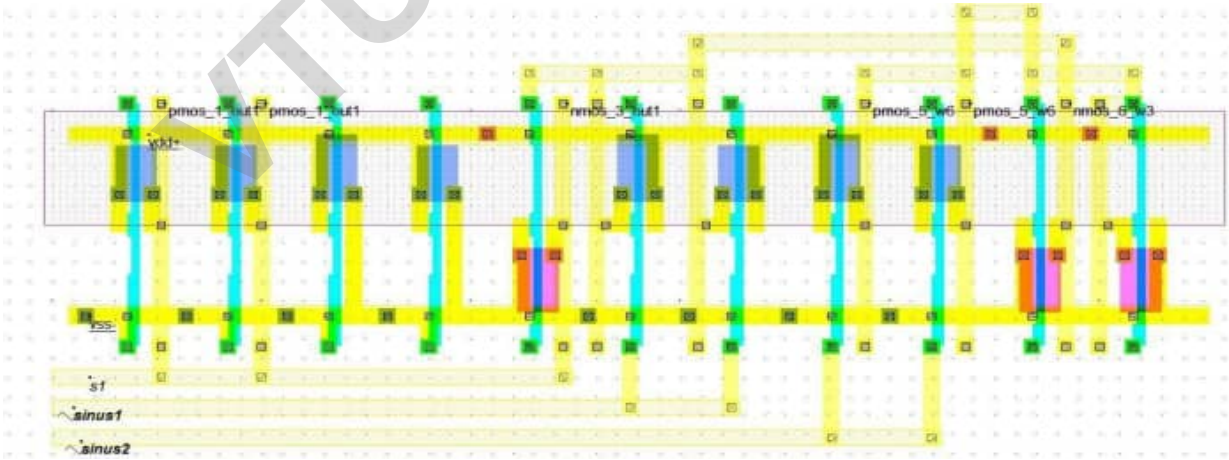
I). Capture schematic of two-stage operational amplifier and measure the following:

a. UGB

b. dB bandwidth

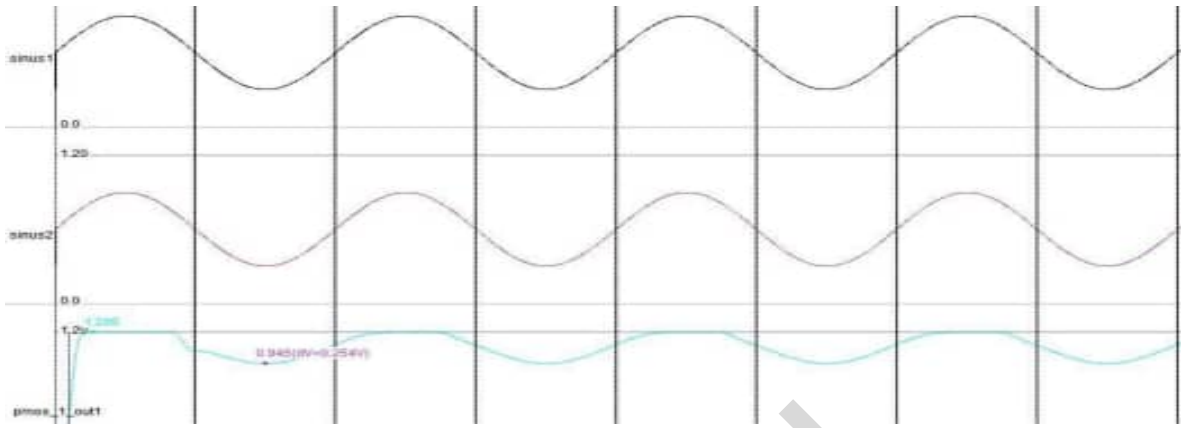


<u>Layout diagram</u>



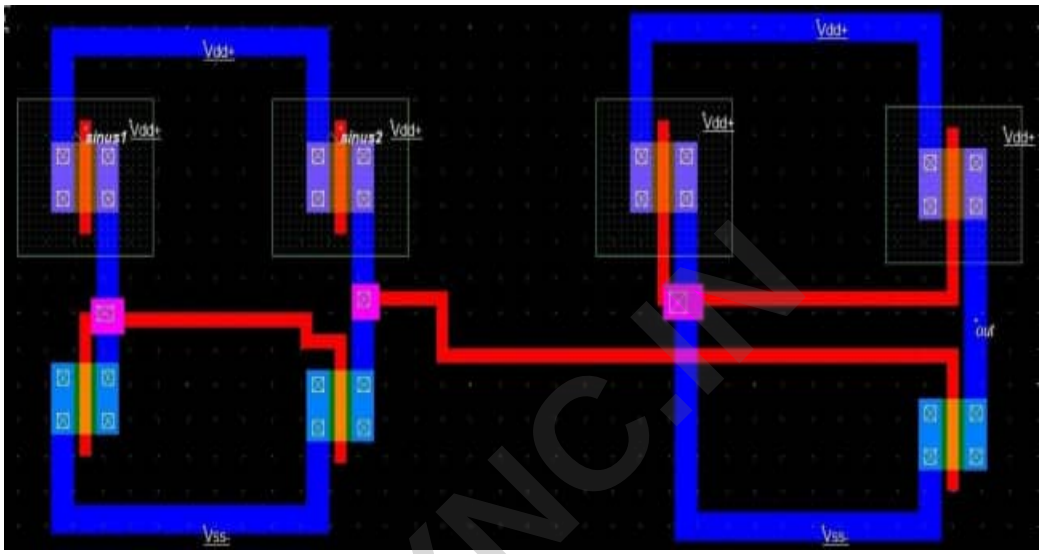
VTUSYNC.IN

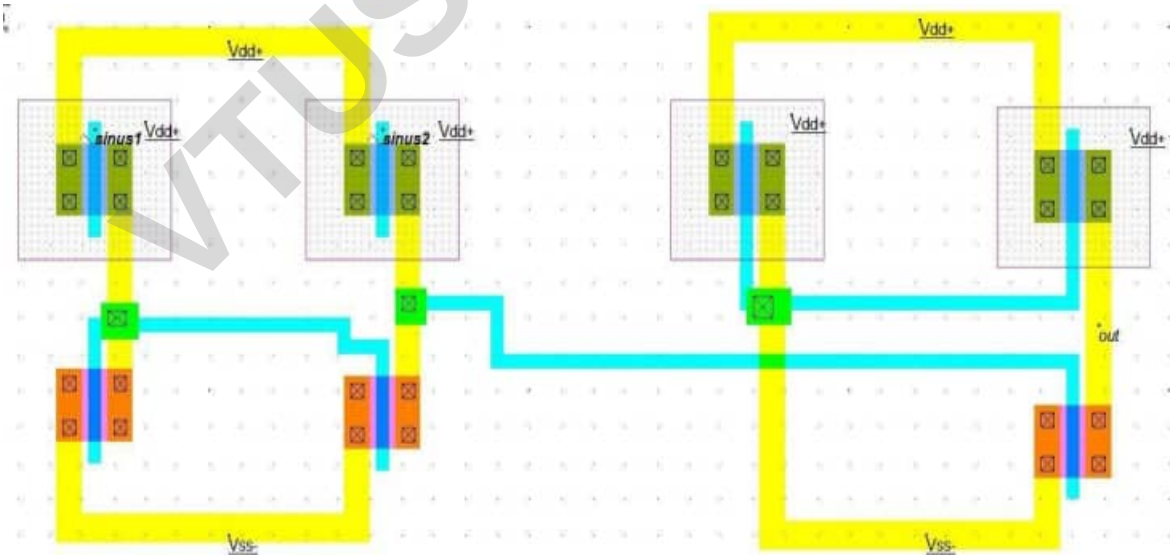
Output waveform



VTUSYNC.IN

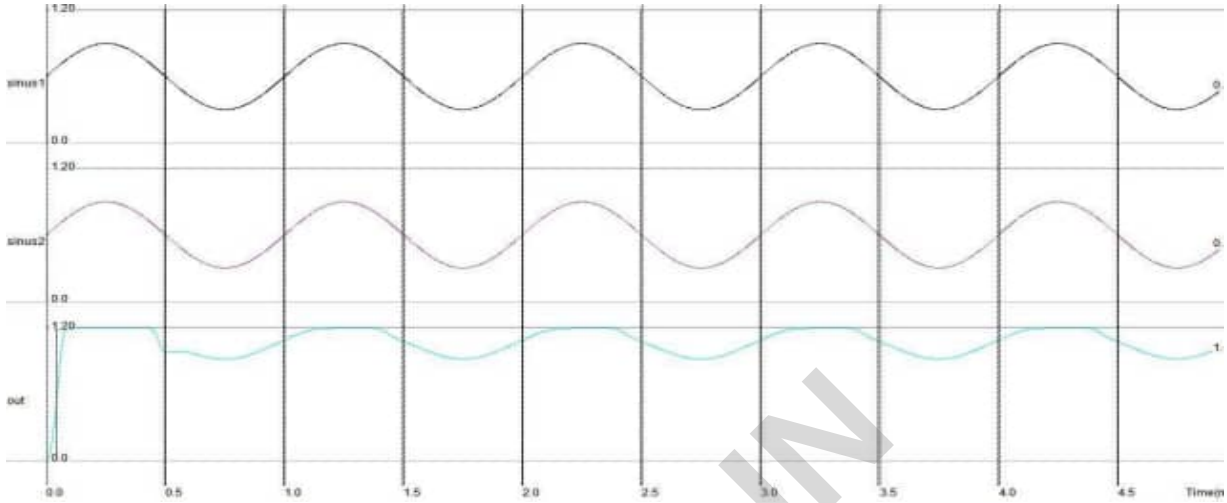
II) Draw layout of two-stage operational amplifier with minimum transistor width set to 300 (in 180/90/45 nm technology), choose appropriate transistor geometries as per the results obtained in 4.a. Use optimum layout methods. Verify for DRC and LVS, extract parasitic and perform post layout simulations, compare the results with pre-layout simulations. Record the observations.





VTUSYNC.IN

Output waveform:



Conclusion:

VTUSYNC.IN